

Shader Review

GLSL Syntax

- GLSL is like C without
 - pointers
 - recursion
 - dynamic memory allocation
- GLSL is like C with
 - Built-in vector, matrix, and sampler types
 - Constructors
 - A great math library

Language features allow us to write concise, efficient shaders.

GLSL Syntax

- GLSL has a preprocessor

```
#ifdef FAST_EXACT_METHOD
    FastExact();
#else
    SlowApproximate();
#endif

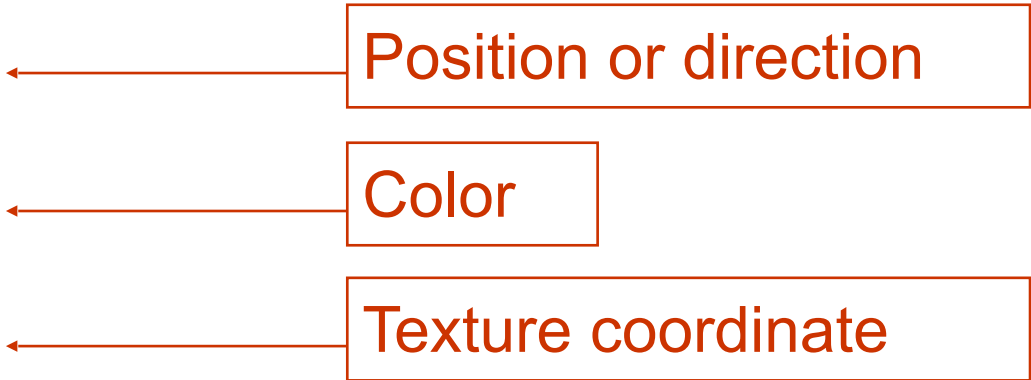
#line 0

// ... many others
```

- All shaders have `main()`

```
void main(void)
{
}
```


GLSL Syntax: Vectors

- Scalar types: `float`, `int`, `uint`, and `bool`
 - Vectors are also built-in types:
 - `vec2`, `vec3`, and `vec4`
 - Also `ivec[2-4]`, `uvec[2-4]`, and `bvec[2-4]`
 - Access components three ways:
 - `.x`, `.y`, `.z`, `.w`
 - `.r`, `.g`, `.b`, `.a`
 - `.s`, `.t`, `.p`, `.q`
- 
- The diagram consists of three horizontal boxes on the right, each containing a label. Red arrows point from each box to the left, towards the component access methods listed in the previous block. The top box is labeled 'Position or direction' and points to the `.x, .y, .z, .w` access method. The middle box is labeled 'Color' and points to the `.r, .g, .b, .a` access method. The bottom box is labeled 'Texture coordinate' and points to the `.s, .t, .p, .q` access method.

GLSL Syntax: Vectors

- Vectors have constructors

```
vec3 xyz = vec3(1.0, 2.0, 3.0);
```

```
vec3 xyz = vec3(1.0); // [1.0, 1.0, 1.0]
```

```
vec3 xyz = vec3(vec2(1.0, 2.0), 3.0);
```

GLSL Syntax: Swizzling

- *Swizzle*: select or rearrange components

```
vec4 c = vec4(0.5, 1.0, 0.8, 1.0);  
  
vec3 rgb = c.rgb;    // [0.5, 1.0, 0.8]  
  
vec3 bgr = c.bgr;    // [0.8, 1.0, 0.5]  
  
vec3 rrr = c.rrr;    // [0.5, 0.5, 0.5]  
  
c.a = 0.5;           // [0.5, 1.0, 0.8, 0.5]  
c.rb = 0.0;          // [0.0, 1.0, 0.0, 0.5]  
  
float g = rgb[1];    // 0.5, indexing, not swizzling
```

- Try it – you'll love it.

GLSL Syntax: Matrices

- Matrices are built-in types:
 - Square: `mat2`, `mat3`, and `mat4`
 - Rectangular: `matmxn`. *m* columns, *n* rows
- Stored *column major*.

GLSL Syntax: Matrices

■ Matrix Constructors

```
mat3 i = mat3(1.0); // 3x3 identity matrix
```

```
mat2 m = mat2(1.0, 2.0, // [1.0 3.0]  
              3.0, 4.0); // [2.0 4.0]
```

■ Accessing Elements

```
float f = m[column][row];
```

```
float x = m[0].x; // x component of first column
```

```
vec2 yz = m[1].yz; // yz components of second column
```

Treat matrix as array of column vectors

Can swizzle too!

GLSL Syntax: Vectors and Matrices

- Matrix and vector operations are easy and fast:

```
vec3 xyz = // ...
```

```
vec3 v0 = 2.0 * xyz; // scale  
vec3 v1 = v0 + xyz;  // component-wise  
vec3 v2 = v0 * xyz;  // component-wise
```

```
mat3 m = // ...  
mat3 v = // ...
```

```
mat3 mv = v * m;      // matrix * matrix  
mat3 xyz2 = mv * xyz; // matrix * vector  
mat3 xyz3 = xyz * mv; // vector * matrix
```

GLSL Syntax: *in* / *out* / *uniform*

■ Recall:

```
uniform mat4 u_modelViewProjection;  
in vec4 position;  
in vec3 color;  
out vec3 fs_color;
```

uniform: shader input constant across
glDraw*

in: shader input varies per vertex
attribute

```
void main(void)  
{  
    fs_color = color;  
    gl_Position = u_modelViewProjection * position;  
}
```

out: shader output

GLSL Syntax: Samplers

- *Opaque* types for accessing textures

```
uniform sampler2D diffuseMap; // 2D texture
```

```
vec3 color = texture2D(diffuseMap, vec2(0.5, 0.5)).rgb;
```

```
// Also samplerCube.
```

GLSL Syntax: Samplers

- *Opaque* types for accessing textures

```
uniform sampler2D diffuseMap; // 2D texture  
  
vec3 color = texture2D(diffuseMap, vec2(0.5, 0.5)).rgb;  
  
// Also samplerCube.
```

Samplers must be uniforms

GLSL Syntax: Samplers

- *Opaque* types for accessing textures

```
uniform sampler2D diffuseMap; // 2D texture
```

```
vec3 color = texture2D(diffuseMap, vec2(0.5, 0.5)).rgb;
```

```
// Also samplerCube.
```

texture() returns a vec4;
extract the components you
need

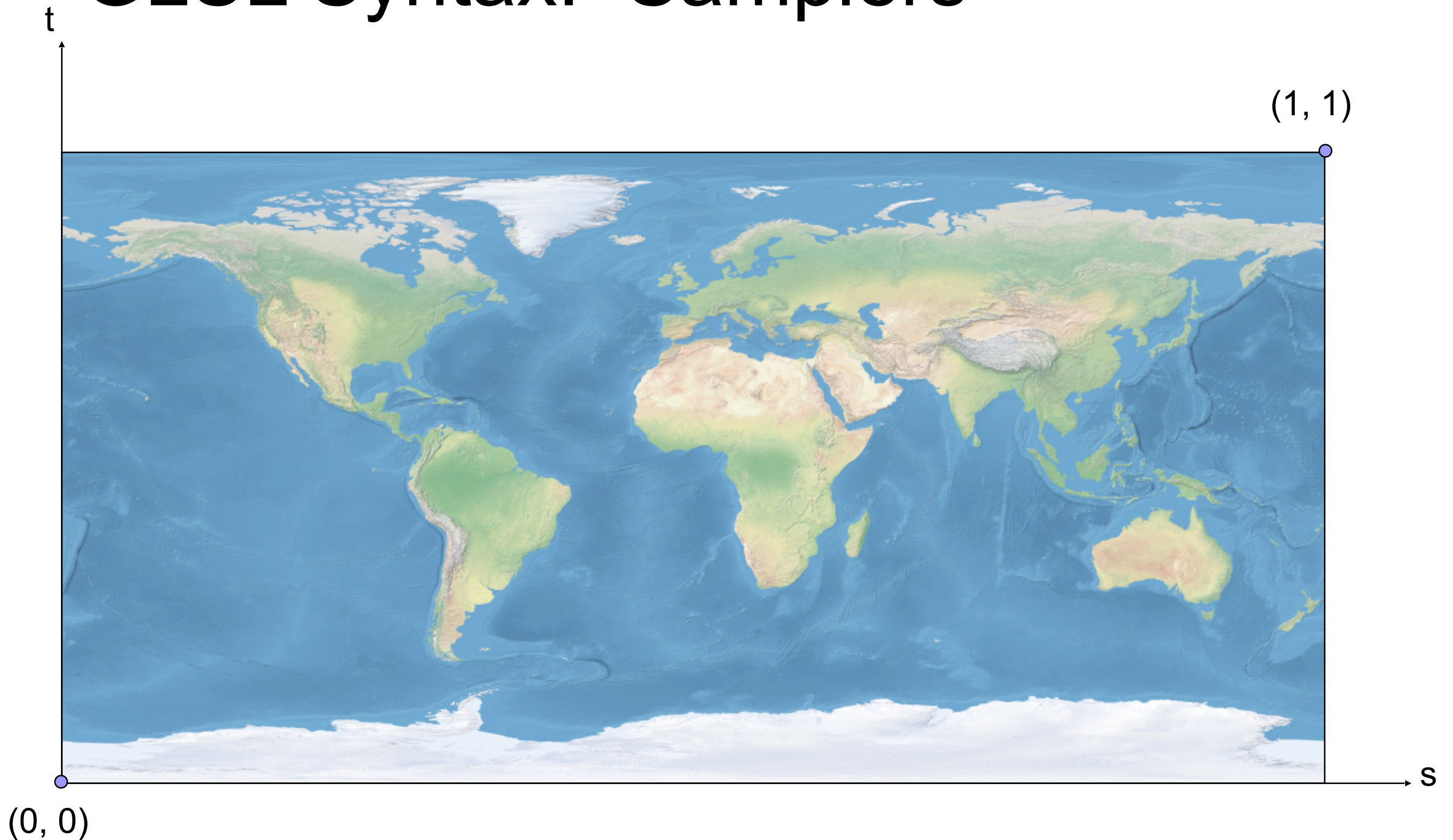
2D texture uses 2D texture
coordinates for lookup

GLSL Syntax: Samplers

- Textures

- Usually, but not always:
 - Textures are square, e.g., 256x256
 - Dimensions are a power of two
- Coordinates are usually normalized, i.e., in the range [0, 1]
- *Texel*: a pixel in a texture
- `texture2D` () does filtering using fixed function hardware

GLSL Syntax: Samplers



GLSL Built-in Functions

■ Selected Trigonometry Functions

```
float s = sin(theta);  
float c = cos(theta);  
float t = tan(theta);
```

```
float as = asin(theta);  
// ...
```

```
vec3 angles = vec3(/* ... */);  
vec3 vs = sin(angles);
```

Works on vectors
component-wise.

GLSL Built-in Functions

■ Exponential Functions

```
float xToTheY = pow(x, y);  
float eToTheX = exp(x);  
float twoToTheX = exp2(x);  
  
float l = log(x); // ln  
float l2 = log2(x); // log2  
  
float s = sqrt(x);  
float is = inversesqrt(x);
```

← One GPU instruction!

GLSL Built-in Functions

■ Selected Common Functions

```
float ax = abs(x); // absolute value
float sx = sign(x); // -1.0, 0.0, 1.0

float m0 = min(x, y); // minimum value
float m1 = max(x, y); // maximum value
float c = clamp(x, 0.0, 1.0);

// many others: floor(), ceil(),
// step(), smoothstep(), ...
```

GLSL Built-in Functions

■ Selected Geometric Functions

```
vec3 l = // ...
vec3 n = // ...
vec3 p = // ...
vec3 q = // ...

float f = length(l); // vector length
float d = distance(p, q); // distance between points

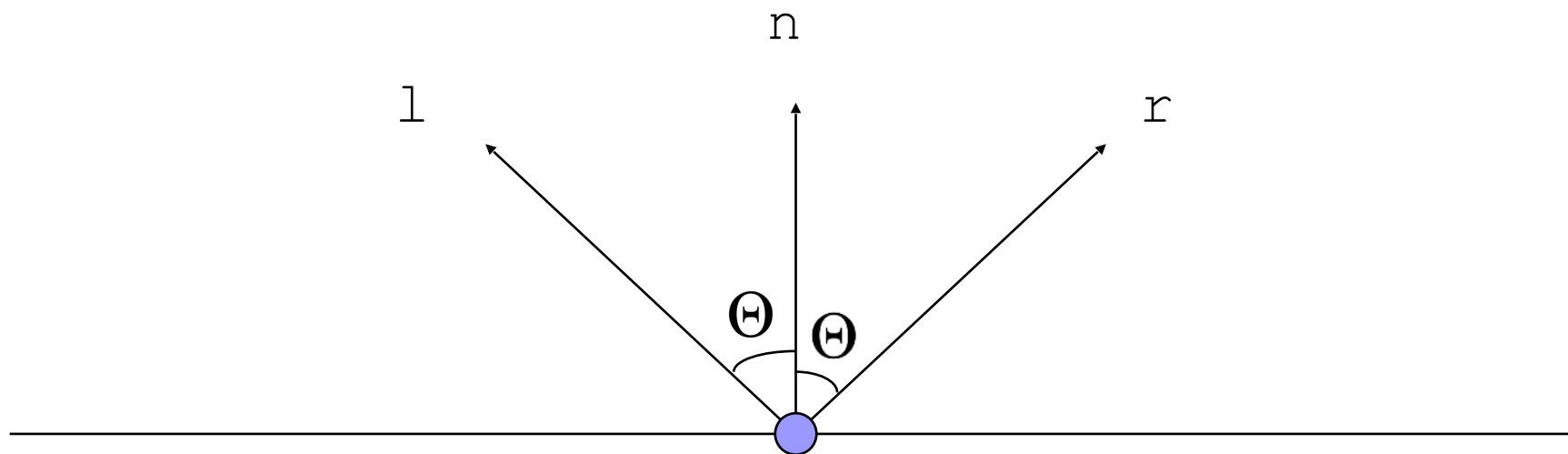
float d2 = dot(l, n); // dot product
vec3 v2 = cross(l, n); // cross product
vec3 v3 = normalize(l); // normalize

vec3 v3 = reflect(l, n); // reflect

// also: faceforward() and refract()
```

GLSL Built-in Functions

- `reflect(-l, n)`
 - Given l and n , find r . Angle in equals angle out



GLSL Built-in Functions

■ Selected Vector Relational Functions

```
vec3 p = vec3(1.0, 2.0, 3.0);  
vec3 q = vec3(3.0, 2.0, 1.0);  
  
bvec3 b = equal(p, q);           // (false, true, false)  
bvec3 b2 = lessThan(p, q);       // (true, false, false)  
bvec3 b3 = greaterThan(p, q);    // (false, false, true)  
  
bool b4 = any(b);                // true  
bool b5 = all(b);                // false
```


GLSL Built-in Functions

- Rewrite with one function call

```
float minimum = // ...  
float maximum = // ...  
float x = // ...  
  
float f = min(max(x, minimum), maximum);
```


GLSL Built-in Functions

- Rewrite this without the `if` statement

```
float x = // ...  
float f;  
  
if (x > 0.0)  
{  
    f = 2.0;  
}  
else  
{  
    f = -2.0;  
}
```

GLSL Built-in Functions

- Rewrite this without the `if` statement

```
float root1 = // ...  
float root2 = // ...  
  
if (root1 < root2)  
{  
    return vec3(0.0, 0.0, root1);  
}  
else  
{  
    return vec3(0.0, 0.0, root2);  
}
```

GLSL Built-in Functions

- Rewrite without `length`.

```
vec3 p = // ...  
vec3 q = // ...
```

```
vec3 v = length(p - q);
```

GLSL Built-in Functions

- What is wrong with this code?

```
vec3 n = // ...
```

```
normalize(n);
```

GLSL Built-in Functions

- Rewrite this in one line of code

```
bool foo(vec3 p, vec3 q)
{
    if (p.x < q.x)
    {
        return true;
    }
    else if (p.y < q.y)
    {
        return true;
    }
    else if (p.z < q.z)
    {
        return true;
    }
    return false;
}
```